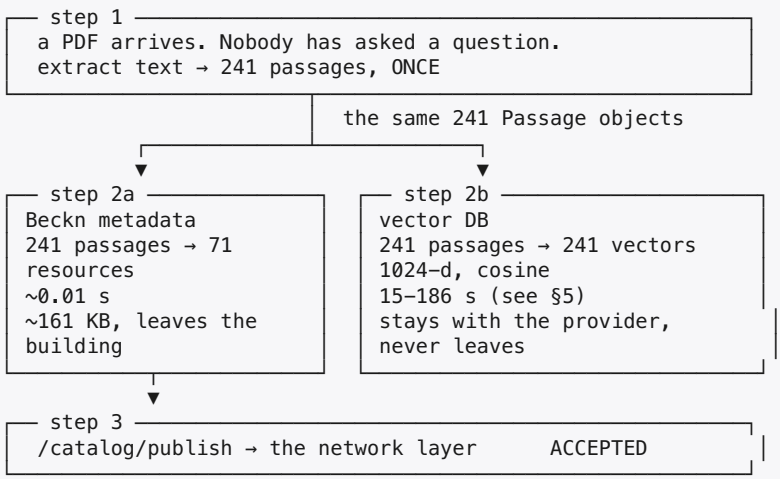
Step 2, walked through

Audience: someone who needs to understand what step 2 does, why it is two branches, and what technology is inside each — without reading the code first.

Every number and every JSON fragment below is **copied from a real run** on IMD's Karnataka agromet bulletin 69/2026 (53 pages). Nothing here is invented for illustration. Re-run it yourself with:

```
.venv/bin/python main.py --all --fresh
```

0. Where step 2 sits



The one sentence that explains the whole shape: the network layer can hold the index card but not the bulletin, so metadata goes out and text stays home.

1. Why two branches and not one index

The two outputs answer two different questions, for two different audiences, with two different lifetimes.

	2a — Beckn metadata	2b — vector DB
answers	"**who** can help me?"	"**what** should I do?"
audience	the network layer, and every consumer app on it	this provider's own node, on direct request
size	161 KB for three states	510 vectors × 1024 floats
leaves the building?	**yes** — published	**no** — never published
goes stale?	slowly (coverage changes rarely)	n/a — read live at answer time
contains advisory text?	**never**	that is its entire job

The decisive constraint is **caching**. A published catalogue is copied and cached by every consumer that fetches it. If you put "spray NAA @ 0.5 ml/lit" inside it, then when next week's bulletin changes that advice, every cached copy is now wrong and there is no mechanism to recall them. Advice must be fetched live, from the provider, at the moment it is asked for — which is what 2b exists to make possible.

So: 2a is an **index card**; 2b is the **bulletin on a shelf**.

2. Step 1 first, because both branches depend on it being one pass

`ingest/passages.py` produces a list of `Passage` objects. Each one carries **both** the text and the resolved metadata:

```
@dataclass(frozen=True)
class Passage:
    text: str                      # → 2b embeds this
    document: str; page: int        # → provenance, cited to a user
    language: str                   # → 2a languages[],  2b filter
    subjects: tuple[Subject, ...]   # → 2a agricultureSubjects[], 2b filter
    area: Area                      # → 2a coverageAreas[], 2b filter
    also_covers: tuple[Area, ...]
    category: str                   # → 2a @type
    topics: tuple[str, ...]
    weather_parameters: tuple[str, ...]
    resource_id: str                # ← THE JOIN between the two branches
    point_id: str
```

2.1 What step 1 refuses, and what it repairs

Three things happen before a single passage exists, and each one changes what the two branches can honestly claim.

A document is either read or refused — never partially trusted. Ingestion goes through MuPDF, so PDF, DOCX, TXT, HTML, EPUB and XPS all read; a DOCX bulletin reaches the catalogue exactly as a PDF does, and a test covers it. Anything no extractor opens — the legacy `.doc` binary, a spreadsheet, an archive — raises `UnusableDocument`, as does a scan with no text layer and a bulletin from a state the vocabulary does not cover (`UnknownState`). All three print a `REFUSED` line naming the reason and let the run continue to the next document. None of them can produce a resource claiming coverage nobody read.

A mis-mapped font is repaired, but only when that provably helps. The Rajasthan bulletin's producer swapped `ब` and `ि` in the font's ToUnicode table and emitted the i-matra at the position it is *drawn* rather than in logical order, so `सिरोही` arrived as `बसरोही` and `बैंगन` as `िबैंगन`. Common function words were untouched — which is why the encoding check called the file healthy — but crop and district names were not, and those are precisely the words that decide what the catalogue can advertise.

`repair_devanagari()` undoes it; `repair_encoding()` decides whether to believe the result by scoring both versions against the crop and district vocabulary:

```
Rajasthan   28 → 37 known terms   applied
UP         121 → 78 known terms   refused (different defect; the same
                                  transform would destroy this file)
Karnataka    0 →  0               refused (no Devanagari)
```

The catalogue for Rajasthan went from advertising 4 crops to 8, from text it always held. A repair that fires prints `encoding fix applied — …` on the run.

Passages are cut on crop boundaries, not only on length. An advisory table runs one crop's advice straight into the next with no blank line, so a size-only split produced passages about two crops and therefore precisely about neither. `_split_on_crop_change()` starts a new passage at a line naming crops the current run does not share; a line naming no crop continues the run, since table rows rarely repeat their crop; and a run too short to survive `MIN_PASSAGE_CHARS` is folded back into its neighbour rather than dropped.

Measured across the three bulletins: passages 357 → 510, passages carrying a resolved subject 208 → 344, passages carrying more than one crop 133 → 68, and passages placed in a district 243 → 330. Text coverage is unchanged at 99.5% (per bulletin: 99.5% / 99.7% / 97.2%). Retrieval barely moved (14 farmer questions: 12 unchanged, 1 better, 1 worse) — **the gain is in labelling, not ranking**, and §4.9 records that honestly.

Why this matters more than it looks

If 2a and 2b each ran their own extraction, they would drift apart over time. The failure mode is not cosmetic:

> Discovery tells a consumer "resource `res-…-crop-in-ka-koppal` covers red gram in Koppal". The consumer asks the provider for detail *inside that resource*. The provider filters its vectors by that resource id — and finds nothing, because its own extractor had tagged those passages differently.
>
> The provider would be failing to answer a question about a resource it had just advertised.

Both branches therefore read the **same method**, `Passage.facets()`. Parity is structural rather than maintained by discipline, and `tests/test_v4.py::test_facet_parity` asserts it for every passage.

A real passage, from page 16

```
Redgram
Flowering
 Nipping in Pigeon pea at 50 days after sowing
to enhance branches, pods per plant and yield.
 Dropping of flower in pigeon pea due to
continuous cloudy condition go for spraying of
NAA @ 0.5 ml/lit of water
 Spraying of pulse magic (10 g/lit of water) to
pigeon pea at flowering stage to enhance yield
...
```

`Passage.facets()` for it — this exact dict is what both branches consume:

```
{
  "language": "en",
  "category": "Crop",
  "area_code": "IN-KA-KOPPAL",
  "area_level": "District",
  "area_name": "Koppal",
  "also_area_codes": [],
  "subject_uris": [
    "https://taxonomy.openagrinet.global/crop/field-pea",
    "https://taxonomy.openagrinet.global/crop/groundnut",
    "https://taxonomy.openagrinet.global/crop/red-gram"
  ],
  "topics": ["NutrientManagement", "PestManagement", "SoilMoisture",
             "Sowing", "Spraying", "WeedManagement"],
  "weather_parameters": [],
  "resource_id": "res-imd-agromet-agriculture-crop-in-ka-koppal"
}
```

Keep that passage in mind. It contains the literal string `spraying of NAA @ 0.5 ml/lit of water`. Watch where that text does and does not end up.

How the facets were resolved — rules, not a model

facet	how	example
`area_code`	the section heading `Agromet Advisory for Koppal district` was seen on page 14 and is tracked forward across pages	`IN-KA-KOPPAL`
`subject_uris`	alias table: `"pigeon pea"`, `"redgram"`, `"tur"`, `"arhar"`, `"अरहर"` all → slug `red-gram`	`.../crop/red-gram`
`topics`	alias table: `"spraying"` → `Spraying`, `"intercultivation"` → `WeedManagement`	6 topics
`category`	a crop subject resolved → `Crop` (a livestock subject or the `LivestockCare` topic → `Livestock`; nothing but weather → `Weather`)	`Crop`
`language`	Unicode block counting; Devanagari/Kannada/Gujarati each have their own block	`en`

Four properties of doing this with rules rather than an LLM:

- **deterministic** — the same PDF resolves identically every run, so re-publishing is a genuine no-op instead of a churning catalogue
- **free** — no inference cost per passage
- **auditable** — every resolution traces to one alias in one JSON file
- **honest** — a term the rules cannot resolve stays unresolved and is *counted* (Karnataka: 83.4% of passages resolved; Rajasthan: 24.1%)

No code path may mint a subject URI. `validate.py` re-checks every `subjectId` in the finished payload against the vocabulary, so even a future LLM enrichment step could only ever select an existing URI, never invent one. A wrong URI is worse than a missing one: it routes a farmer to a provider that cannot help, and nothing downstream can tell.

---

3. Branch 2a — building the Beckn metadata

3.1 The unit of publication is a capability, not a document

Passages are grouped by **(primary area × category)**:

```
241 passages  ──group──▶  71 resources
```

One resource is "*crop advisory for Koppal*" — 8 passages from pages 15–17. Not one resource per PDF, and not one per advisory line.

This is a product decision with a technical consequence. A farmer does not look for "bulletin 69/2026"; they look for crop advice for their district. So the resource id is a pure function of *(provider, domain, category, area)*:

```
res-imd-agromet-agriculture-crop-in-ka-koppal
    └ provider ┘ └ domain ┘  └cat┘ └── area ──┘
```

Nothing about the bulletin issue appears in it. **Next Friday's bulletin 70/2026 therefore updates these same 71 resources instead of creating 71 more.** If the id carried the issue number, every publish would duplicate the catalogue and every consumer's cache would fill with near-identical resources.
(`test_resource_id_is_stable_and_document_independent`)

3.2 `@type` follows the category, not the document

Because the unit is *(area × category)*, one bulletin publishes **four different capability types**:

capability type	resources (Karnataka)
`CropAdvisoryCapability`	22
`LivestockAdvisoryCapability`	14
`HorticultureAdvisoryCapability`	13
`WeatherAdvisoryCapability`	12

And each type carries **only the attributes it is for**:

- a crop resource has `agricultureSubjects`, and **no** `weatherParameters`
- a forecast resource has `weatherParameters`, and **no** `agricultureSubjects`

`validate.py` fails any payload where those disagree (`test_capability_type_governs_attribute_groups`). This is what stops a resource from advertising a capability it has no data behind — a forecast resource claiming to know about sugarcane would be discoverable for sugarcane questions it cannot answer.

> **A bug this caught during development.** The Karnataka bulletin says "*Animals should be vaccinated against FMD, Enterotoximea, Black quarter*" — which names no animal in the vocabulary, but does mention rain. It was therefore filed as a **Weather** resource: a livestock advisory advertised as a forecast. The fix was to let the `LivestockCare` topic decide the category when no species resolves. 7 of the 8 vaccination passages now type correctly.

3.3 `resourceAttributes` — where all the domain work lives

Beckn core defines `Resource.resourceAttributes` as an **open JSON-LD object**:

```yaml
Attributes:
  required: ['@context', '@type']
  additionalProperties: true      # ← the entire extension point of Beckn v2
```

The spec requires exactly two keys and then deliberately gets out of the way. Everything agriculture-specific lives in that space. Here is the **real, published object** for the Koppal crop resource (subjects trimmed for length):

```jsonc
{
  "@context": "https://schemas.openagrinet.global/schema/CropAdvisoryCapability/v0.1/context.jsonld",
  "@type": "openagrinet:CropAdvisoryCapability",
  "subjectCategories": ["Crop"],
  "languages": ["en"],
  "coverageAreas": [
    { "codeScheme": "OPENAGRI-DISTRICT", "areaCode": "IN-KA-KOPPAL",
      "areaLevel": "District", "areaName": "Koppal" }
  ],
  "geographicGranularity": "District",
  "agricultureSubjects": [
    { "subjectId": "https://taxonomy.openagrinet.global/crop/cotton",
      "subjectType": "Crop", "descriptor": { "code": "COTTON", "name": "Cotton" } },
    { "subjectId": "https://taxonomy.openagrinet.global/crop/field-pea",
      "subjectType": "Crop", "descriptor": { "code": "FIELD_PEA", "name": "Field pea" } }
    /* …+7 more: cowpea, groundnut, maize, onion, paddy, pomegranate, red-gram */
  ],
  "topics": ["Irrigation", "NutrientManagement", "PestManagement",
             "SoilMoisture", "Sowing", "Spraying", "WeedManagement"],
  "evidence": {
    "sourceDocuments": ["imd_karnataka_agromet.pdf"],
    "passageCount": 8,
    "pageRange": [15, 17]
  },
  "updateFrequency": "P1D"
}
```

Now compare that against the raw passage in §2. The passage said `spraying of NAA @ 0.5 ml/lit of water`. The published object says `"topics": [… "Spraying" …]`.

That is the whole discipline in one line: *the fact that this provider gives spraying advice for Koppal* is metadata and is publishable. *What to spray* is an answer, and it is not.

The rule for deciding what belongs in here: **a field belongs if a consumer could plausibly discover on it.** "Which crops?" yes. "Which districts?" yes. "What should I spray on Tuesday?" no.

The `evidence` block is provenance of the *claim*, not the content of it: a consumer deciding whether to trust this provider can see the resource rests on 8 passages across 3 pages, without receiving any of the text.

3.4 Three checks that are ours, not the spec's

Closed-vocabulary membership. Every `topic` must *be* a topic from `capabilities.json`; every `subjectId` must be one the taxonomy minted; every `codeScheme` one of two known schemes. This is strictly stronger than scanning those fields for prose — and it matters for a subtle reason: `"Spraying"` is simultaneously a canonical topic name *and* a word that any prose detector would flag. Membership resolves that cleanly where a regex could not.

No prose. Every remaining string is scanned for advisory markers (dosages, `ml/l`, `@ 4`, imperatives like "apply"/"carry out") and for sentence-length runs. If prose appears anywhere, the publish is refused rather than

downgraded. `test_no_advisory_text_in_payload` greps the finished 161 KB payload for real sentences taken from the source bulletins — `"NAA"`, `"moisture stress"`, `"ml/litre"` — and all three are absent.

Declaration matches contents. `subjectCategories` must include every `subjectType` the resource actually carries. This one was added *because it caught a live bug*, and it is a good example of why the per-document JSON view (`evidence/resources/*.json`) is worth having:

> 11 of Karnataka's 14 Livestock resources declared `subjectCategories: ["Livestock"]` while their `agricultureSubjects` contained **Crop**-typed subjects — cowpea, onion, paddy, the fodder crops those advisories name alongside the animals. Self-contradictory metadata: a consumer filtering `subjectCategories == "Crop"` would have skipped a resource that genuinely contains crops.
>
> `subjectCategories` is now the union of the capability's declared categories and the subject types present, so `livestock-in-ka-bidar` declares `["Crop", "Livestock"]`. Re-audit: 0 of 84 incoherent.

The aggregate statistics could never have shown this. Reading one document's resources as JSON showed it immediately.

3.5 The envelope

```
{
  "context": {
    "domain": "agriculture",
    "action": "publish",
    "version": "2.0.0",
    "transactionId": "…uuid4…",
    "messageId": "…uuid4…",
    "timestamp": "2026-09-03T…Z",
    "schemaContext": [
      ".../CropAdvisoryCapability/v0.1/context.jsonld#openagrinet:CropAdvisoryCapability",
      ".../HorticultureAdvisoryCapability/…",
      ".../LivestockAdvisoryCapability/…",
      ".../WeatherAdvisoryCapability/…"
    ]
  },
  "message": {
    "catalogs": [ /* 3 — one per state */ ],
    "publishDirectives": [
      { "catalogId": "cat-imd-agromet-agriculture-in-ka",
        "catalogType": "REGULAR", "visibleTo": ["local-network"] }
    ]
  }
}
```

`schemaContext` lists exactly the capability types present, which is what tells the receiver which JSON-LD vocabularies it needs to interpret what it is indexing.

Two honest deviations, both inherited from the target `message_update.json` rather than chosen here:

1. `context.domain` **is not in Beckn v2.** The v2 `Context` schema has no `domain` property — v2 replaced it with `schemaContext`. It is tolerated (Context is not `additionalProperties: false`) but it is a house extension.
2. `CatalogPublishAction` is marked `deprecated: true` in `core-v2.0.0-lts`, even though `/catalog/publish` is still the documented publish path.

---

4. Branch 2b — the vector DB, in full technical detail

This is the branch that costs essentially all of the wall clock, and the one that makes later follow-up questions answerable.

4.1 The embedding model

property	value	why it matters
model	**`intfloat/multilingual-e5-large`**	same family as production's `hf/multilingual-e5-large` in Marqo, so behaviour transfers
parameters	~560 M	this is why 2b takes seconds, not milliseconds
dimensions	**1024**	the collection is created to match; a mismatch is refused, not coerced
distance	**cosine**	vectors are L2-normalised at encode time, so cosine is a dot product
languages	~100, incl. Hindi, Kannada, Gujarati, Marathi, Tamil, Telugu	**one index serves all three bulletins** — no per-language index
context window	**512 tokens — truncates silently**	see 4.2
device	auto: `cuda` → `mps` → `cpu`	measured on Apple Silicon `mps`; see §5 for why the timing varies

The asymmetric prefixes. e5 was trained with two literal string prefixes, and they are not decoration:

```
E5_PASSAGE_PREFIX = "passage: "     # prepended at INDEX time
E5_QUERY_PREFIX   = "query: "       # prepended at SEARCH time
```

Omitting them, or swapping them, measurably degrades retrieval. Because it is easy to forget, index and query are **separate methods** — `embed_passages()` and `embed_query()` — rather than one method with a flag a caller can get wrong.

4.2 The 512-token limit, and why passages are capped at 700 characters

e5-large accepts 512 tokens and **discards everything beyond that with no error** — you simply get a vector computed from a prefix of your passage, which is indistinguishable from a correct one if you only look at the output.

This bites unevenly across languages, which is exactly the kind of thing that would go unnoticed in a demo built on English documents only:

> Devanagari tokenises roughly 2–3× less efficiently than Latin for the same character count. A character limit that is safe for the Karnataka bulletin would silently truncate the Hindi ones.

So `MAX_PASSAGE_CHARS = 700`, and every run *measures* rather than assumes:

```
Karnataka   tokens: max=254 mean=105 — all 241 passages fit inside the 512-token window
UP          tokens: max=299 mean=146 — all 240 passages fit inside the 512-token window
Rajasthan   tokens: max=314 mean=178 — all  29 passages fit inside the 512-token window
```

Note UP and Rajasthan run ~25% higher tokens-per-character than Karnataka — that is the Devanagari penalty, visible in the numbers.

4.3 The collection

```
create_collection(
    collection_name="agri_passages_v4",
    vectors_config=VectorParams(size=1024, distance=Distance.COSINE),
)
```

Qdrant in embedded mode — `QdrantClient(path=…)` runs the engine *inside the Python process* against a local directory. No server, no Docker, no port. Set `QDRANT_URL` to a URL to talk to a real cluster; nothing else in the code changes.

4.4 What is stored per passage

One point per passage. 510 points for the three bulletins.

payload field	purpose
(the vector)	1024 floats — what similarity actually runs on
`text`	returned verbatim as the answer at follow-up time
`document`, `page`	provenance — an answer cites `imd_karnataka_agromet.pdf p.16`
`resource_id`	**the join back to branch 2a**
`area_code`, `also_area_codes`	narrow to a district or state
`language`	answer a farmer in the language they asked in
`category`	crop vs weather vs livestock vs horticulture
`subject_uris`, `topics`, `weather_parameters`	the same facets 2a published

Everything after `text/document/page` comes from `**p.facets()` — literally the same call 2a makes.

4.5 `resource_id` is the load-bearing field

Here is why it exists, and it is the crux of the whole two-branch design:

```
1. consumer → network:  "who covers red gram in Koppal, in English?"
2. network  → consumer:  matches on resourceAttributes ONLY (it holds nothing else)
                        → "res-imd-agromet-agriculture-crop-in-ka-koppal"
3. consumer → provider:  (directly, bypassing the network)
                        "what do I do about flower drop?" + that resource id
4. provider → vector DB: search, FILTERED to resource_id ∈ {that id}
                        → the passage, with its page number
```

Step 4 is only possible because the vector payload carries the same `resource_id` the catalogue published. `test_retrieval_can_be_scoped_to_advertised_resources` walks exactly this path: discover → collect ids → filtered search → assert every hit is inside an advertised resource.

⚠ **It scopes, it does not authenticate.** Resource ids are public. Filtering by them restricts *where* the search looks; it does not prove the caller was ever given them. Real Beckn signatures are not implemented — the largest gap in this package.

4.6 Deterministic point ids

```
point_id = uuid5(NAMESPACE, f"{provider}:{document}:{page}:{ordinal}")
# page 16 passage above → db5e593f-7a80-5a69-8b21-f4f82824ebc6
```

Notably **not** a hash of the text. Re-onboarding the same PDF therefore *upserts* the same 241 points rather than appending a second copy of every passage — and an edited passage updates in place instead of leaving its stale predecessor in the index. Verified by `test_reingest_is_idempotent`: the count before and after a second run is identical (510 → 510).

4.7 Filters

Payload indexes are declared on `resource_id`, `area_code`, `language`, `category`, `document`.

⚠ **Embedded Qdrant ignores payload indexes and evaluates filters by scanning.** Results are correct — but a latency figure measured here must not be quoted as a production one. `VectorIndex.describe()` prints which mode it is in for exactly this reason. That honesty is the point: at 510 passages scanning is free; at a few million it is not.

4.8 It actually retrieves

```
query: "pigeon pea flowers dropping, what should I spray"

0.892  imd_up_agromet.pdf        p.39  res-…-crop-in-up
       "Pigeon Pea (Flowering) At flowering, scout for Helicoverpa larvae and
        damaged flowers… spray Emamectin benzoate 5 SG @ 200 g/ha…"
0.892  imd_karnataka_agromet.pdf p.16  res-…-crop-in-ka-koppal
0.851  imd_karnataka_agromet.pdf p.13  res-…-crop-in-ka-kalaburagi
```

And the semantic claim, tested rather than asserted — the word **"tur" never appears in the bulletin** (it says "Redgram"/"Pigeon pea"):

```
query: "my tur crop flowers are dropping"  →  finds the same page-16 passage
```

Cross-script too — a Hindi query returns Hindi passages from the UP bulletin:

```
query: "धान की फसल में सिंचाई"   (paddy crop irrigation)
0.848  imd_up_agromet.pdf p.22  lang=hi
0.828  imd_up_agromet.pdf p.17  lang=hi
```

That is what the 1024-d multilingual model buys, and it is the thing the `lexical` fallback cannot do — which is why every result from that fallback is stamped `semantic=False`.

4.9 One measured caveat on ranking

Scoped to the 8 passages of the Koppal crop resource, asking for the flower-drop advisory three ways:

query	rank of the correct passage	top score
`"pigeon pea flower drop remedy"`	**1**	0.876
`"flower dropping in redgram due to cloudy weather"`	**1**	0.805
`"my tur crop flowers are dropping, what do I do?"`	**3**	0.823 (correct one: 0.810)

So the paraphrase does work — "tur" never appears in the bulletin and the right passage is still retrieved — but it **drops to rank 3**, and the top three scores sit within 1.6% of each other (0.823 / 0.819 / 0.810). Two consequences worth stating rather than discovering later:

- return **several** passages to the answering layer, not just the top one
- a re-ranking step (or a cross-encoder) is the obvious next improvement, and is what production would add before trusting rank 1

This is a single spot measurement on one resource, not a benchmark. No recall@k number should be quoted from it.

A wider measurement, run when the crop-boundary chunking of §2.1 landed: 14 farmer questions in Hindi and English, across all three bulletins.

Two ground truths were used, and they disagree on the absolute number, which is itself the finding. **Strict** demands one specific passage — the Hindi one from the bulletin the question is about. **Correct-advice** accepts any passage that answers the question, including the English equivalent from another state.

```
                                          before chunking      after
MRR@5, strict ground truth                          0.655      0.643
MRR@5, correct-advice ground truth                     --      0.821  (shipped)
                                                               0.836  + gram alias fix
                                                               0.857  + subject filter (not shipped)

per-question movement (strict):  12 unchanged · 1 better · 1 worse
   better   fall armyworm on maize                 #3 → #2
   worse    "wilt in bengal gram"                  left the top 5
```

Quote neither MRR as a benchmark: 14 questions on three bulletins, written by the same person who read the bulletins. The per-question movement is the part that holds under either scoring.

Two things that measurement exposed, both worth more attention than the MRR:

- **An alias must not be a whole word inside another crop's name.** `gram` was an alias for Bengal gram, and India grows black, green, horse and red gram. Karnataka and UP were publishing Bengal gram coverage from bulletins that never mention chickpea. Removing the alias withdrew both false claims — one fewer subject URI on the network, and a correct one.
- **A subject filter is tempting and was left out.** Every passage already stores `subject_uris`, and an English question resolves through the same taxonomy, so filtering by subject is a two-line change that moved one query from rank 2 to rank 1. It also returns an *empty set* when the query names a crop the corpus does not cover — worse than an imperfect answer — and would need a fallback to be safe. One rank on fourteen queries did not justify it.

An English question, incidentally, does not fail against this corpus. It returns correct English advice — often from a different state's bulletin, since nothing in a bare semantic query says where the farmer is. Scoping to the right provider is discovery's job (`area_code`, then `resource_ids` on the follow-up leg), not retrieval's.

---

5. On "in parallel" — what it does and does not buy

The two branches run concurrently in a two-thread pool. Measured across four separate runs on the same laptop (Apple Silicon, `mps`), fastest to slowest:

document	2a	2b	wall clock
Karnataka	0.002 – 0.018 s	**14.9** / 33.5 / 83.6 / 186.2 s	= 2b
UP	0.001 – 0.11 s	**21.7** / 40.6 / 111.7 / 153.5 s	= 2b
Rajasthan	0.001 – 0.037 s	**3.8** / 4.7 / 21.3 / 48.5 s	= 2b

Two things to take from this table.

First, **2b's absolute timing varies by more than 12× on identical input.** The fastest column is an otherwise-idle machine; the slowest was measured immediately after a full test run, with the GPU hot and contended. Do not quote a single number for it — quote the range or measure it on the target hardware. A dedicated GPU would be both faster and far more consistent.

Second, and unaffected by that variance: **concurrency saves ~0 seconds**, and the run says so out loud rather than rounding it into a win:

```
parallel  2a 0.018s · 2b 83.573s · wall 83.609s
          ─ no measurable saving ─ 2b is 4,541x slower than 2a,
            so wall clock is simply 2b
```

The ratio ranged from **568× to 15,654×** across runs. In every single one, 2a was lost in the noise of 2b.

Branch 2a is dict-building; branch 2b runs a 560M-parameter neural network. There is nothing meaningful to overlap. Reporting a speed-up here would be a lie by rounding.

So why run them in parallel at all? Because the reason is architectural, not throughput:

- neither branch blocks the other, and neither is an input to the other
- either can fail without corrupting the other — a Qdrant outage does not stop a catalogue being published, and a schema rejection does not lose the index
- they have genuinely different lifetimes and owners (network vs provider)
- it keeps the door open to running them on different machines

If you wanted an actual speed-up, it would come from inside 2b — larger batches, a real GPU, or embedding several documents at once — not from running the two branches together.

6. Step 3, briefly

```
target        in-process NetworkNode stand-in
validation    spec valid: 3 catalogue(s), 84 resource(s), 84 resourceAttributes checked
payload       154,599 bytes
ack           ACCEPTED — 3 catalogue(s), 84 resource(s) indexed,
                         123,219 bytes of resourceAttributes held
round-trip    verified — every resourceAttributes field held as published
```

What the network layer now holds, and it is worth reading as a list of what it *can* match a question against:

```
resources 84 · subject URIs 51 · area codes 141 · topics 13
subject categories 4 · weather parameters 7 · languages 2
```

And what it does not hold:

```
advisory text 'NAA'             present in network layer: False
advisory text 'moisture stress' present in network layer: False
advisory text 'ml/litre'        present in network layer: False
```

Stated in the right order: **the network layer holds the complete `resourceAttributes` of all 84 resources — 123 KB of it. What it does not hold is document text.** That single asymmetry is why a consumer has to make two hops, and why this pipeline has two branches.

7. What I would want asked about this

Honest gaps, so they come from me rather than from the room:

1. **No Beckn signatures.** No `AuthorizationHeader`, no `AckSignatureHeader`, no `context.key`. Discovery trusts whoever calls it. Biggest gap.
2. **`network_node.py` is a model, not an implementation** — no registry lookup, no fan-out, persistence is a dict.
3. **The schema URLs do not resolve.** `schemas.openagrinet.global` fails DNS today and `OpenAgriNet/network-specs` holds a 15-byte README. `@context` is required by the spec, so the intended URL is emitted rather than a substitute that happens to resolve.
4. **District codes are not LGD** (the correct national scheme, numeric). We emit `OPENAGRI-DISTRICT` with a readable code instead of a number we cannot vouch for.
5. **Hindi vs Marathi is not distinguished** — they share the Devanagari block. Reported as `hi` with `ambiguous=True` rather than guessed.
6. **A small encoding defect in the source PDFs**: 0.7% of UP's Devanagari words and 0.1% of Rajasthan's are mis-mapped glyphs (प्रदि for प्रदेश). Embedding tolerates it; exact-term lookup on an affected word does not.
7. **Retrieval quality is barely measured.** §4.9 is one spot check on one resource: the correct passage ranks 1st for two phrasings and 3rd for a third, with the top scores within 1.6% of each other. That is enough to say "return several passages, and add a re-ranker" — and nowhere near enough to quote a recall@k. No labelled set, no cross-lingual benchmark.
8. **Rajasthan's catalogue is thin** — 12% subject resolution, 4 resources. The run warns about it. It is a property of that bulletin (mostly weather warnings), not a bug, but it means Rajasthan crop discovery would be poor.

Appendix: reproducing every number here

```
.venv/bin/python main.py --all --fresh
#   → evidence/publish_payload.json      the combined /catalog/publish body
#   → evidence/resources/{karnataka,up,rajasthan}.json   per-bulletin resources
#   → evidence/SCENARIO_1_TRANSCRIPT.txt

.venv/bin/python main.py --all --show-resource livestock-in-ka-koppal
#   → prints one resource's full resourceAttributes to stdout

.venv/bin/python main.py \
       --file imd_karnataka_district_kannada.pdf
#   → REFUSED: a scanned PDF publishes nothing rather than a resource
#     claiming coverage we never read

.venv/bin/python main.py --file <a bulletin from a fourth state>
#   → REFUSED: the vocabulary covers three states, so an area code for a fourth
#     would be invented. The run says so and carries on to the next document.

.venv/bin/python main.py --file <anything.doc>
#   → REFUSED: no extractor opens the legacy .doc binary. DOCX, TXT, HTML, EPUB
#     and XPS do read, and go through exactly as a PDF does.

EMBEDDING_BACKEND=lexical .venv/bin/python main.py --all --fresh
#   → the whole flow in ~0.4s with no model. Branch 2a output is byte-identical;
#     every retrieval result is stamped semantic=False.

.venv/bin/pytest tests/test_v4.py -q -m "not semantic"  # 43 passed, 15s
.venv/bin/pytest tests/test_v4.py -q -m semantic        # 1 passed, 52s
```